Complete mathematical programs for the M_{22} census

Every required mathematical program is printed in full. The census and validator reuse the previously reviewed programs byte-for-byte. The coordinate program extends the earlier shadow proof to rank two and the source's independent endecad complements. The complete labelled-code, literal-configuration and Borel programs are reused; the reconciliation keeps all interpretations separate. The driver and table-formatting script are separately bound auxiliary files, not mathematical premises hidden by excerpts. Catalogue, exact-algorithm and classical group-identification trust remains as stated in the proof. No Lean claim is made.

phase164_m22_census.g

```
# Author census candidate for L073. No source geometry/table data are read.
# Run in a fresh working directory: the exact witnesses are written locally.
LoadPackage("tomlib");;
SizeScreen([100000,24]);;
tom:=TableOfMarks("M22");;G:=UnderlyingGroup(tom);;
ids:=MaximalSubgroupsTom(tom)[1];;ns:=Length(OrdersTom(tom));;inc:=SubsTom(tom);;
derived:=Filtered([1..ns-1],i->not ForAny([i+1..ns-1],j->i in inc[j]));;
if derived<>SortedList(ids) then Error("maximal catalogue mismatch");fi;
maxReps:=List(ids,i->RepresentativeTom(tom,i));;
AA:=AutomorphismGroup(G);;A:=Normalizer(SymmetricGroup(LargestMovedPoint(G)),G);;
if Index(A,Centralizer(A,G))<>Size(AA) then Error("incomplete automorphism image");fi;
M:=Concatenation(List(maxReps,H->Orbit(G,H,OnPoints)));;
if Length(Set(M))<>Length(M) then Error("overlapping maximal classes");fi;
AH:=ActionHomomorphism(A,M,OnPoints);;AP:=Image(AH);;ms:=List(M,Size);;
Print("INPUT G_ORDER ",Size(G)," TOM_CLASSES ",ns," MAXIMAL_ORDERS ",List(maxReps,Size),"
OBJECTS ",Length(M)," AUT_ORDER ",Size(AP),"\n");
meetCache:=[];;rowCache:=[];;
Meet:=function(i,j)
 local a,b;
 a:=Minimum(i,j);b:=Maximum(i,j);
 if not IsBound(meetCache[a]) then meetCache[a]:=[];fi;
 if not IsBound(meetCache[a][b]) then meetCache[a][b]:=Intersection(M[a],M[b]);fi;
 return meetCache[a][b];
end;;
CosetRow:=function(i,j)
 if not IsBound(rowCache[i]) then rowCache[i]:=[];fi;
 if not IsBound(rowCache[i][j]) then
  rowCache[i][j]:=Set(List(RightTransversal(M[j],Meet(i,j)),x->RightCoset(M[i],x)));
 fi;
 return rowCache[i][j];
end;;
PairSignature:=function(t) return [SortedList(ms{t}),Size(Meet(t[1],t[2]))];end;;
pairReps:=[];;pairSigs:=[];;pairTried:=0;;
for singleOrbit in Orbits(AP,[1..Length(M)]) do
 a:=singleOrbit[1];S:=Stabilizer(AP,a);
 for orb in Orbits(S,Difference([1..Length(M)],[a])) do
  t:=Set([a,orb[1]]);sig:=PairSignature(t);pairTried:=pairTried+1;
  bucket:=Filtered([1..Length(pairReps)],j->pairSigs[j]=sig);
  if not ForAny(bucket,j->RepresentativeAction(AP,t,pairReps[j],OnSets)<>fail) then
   Add(pairReps,t);Add(pairSigs,sig);
  fi;
 od;
od;
Print("PAIR_TOTAL TESTED ",pairTried," CLASSES ",Length(pairReps),"\n");
Recognize:=function(t)
 local P,Q,R,B;
 P:=Meet(t[1],t[2]);Q:=Meet(t[1],t[3]);R:=Meet(t[2],t[3]);
```

```
 if Size(ClosureGroup(P,Q))<>ms[t[1]] then return fail;fi;
 if Size(ClosureGroup(P,R))<>ms[t[2]] then return fail;fi;
 if Size(ClosureGroup(Q,R))<>ms[t[3]] then return fail;fi;
 B:=Intersection(P,Q);
 if Length(Intersection(CosetRow(t[1],t[2]),CosetRow(t[1],t[3])))<>Index(R,B) then return
 fail;fi;
 if not ForAll([P,Q,R],H->Index(H,B)>1) then Error("recognition failed firmness
 implication");fi;
 return rec(borel:=B,signature:=[SortedList(ms{t}),SortedList(List([P,Q,R],Size)),Size(B)]);
end;;
triples:=[];;tripleSigs:=[];;borels:=[];;tried:=0;;acceptedBeforeFusion:=0;;
for pidx in [1..Length(pairReps)] do
 pair:=pairReps[pidx];S:=Stabilizer(AP,pair,OnSets);localTried:=0;localAccepted:=0;
 for orb in Orbits(S,Difference([1..Length(M)],pair)) do
  t:=Set(Concatenation(pair,[orb[1]]));tried:=tried+1;localTried:=localTried+1;
  result:=Recognize(t);
  if result=fail then continue;fi;
  acceptedBeforeFusion:=acceptedBeforeFusion+1;localAccepted:=localAccepted+1;
  bucket:=Filtered([1..Length(triples)],j->tripleSigs[j]=result.signature);
  if ForAny(bucket,j->RepresentativeAction(AP,t,triples[j],OnSets)<>fail) then continue;fi;
  Add(triples,t);Add(tripleSigs,result.signature);Add(borels,result.borel);
 od;
 Print("PAIR_STAGE ",pidx," TESTED ",localTried," ACCEPTED ",localAccepted," GLOBAL_CLASSES
 ",Length(triples),"\n");
od;
PrintTo("phase164_m22_witnesses.g","# Independently generated witnesses, not source
input.\nG:=",G,";\nA:=",A,";\nmaximalClassRepresentatives:=",maxReps,";\nM22Pairs:=",List(pa
irReps,t->M{t}),";\nM22Triples:=",List(triples,t->M{t}),";\n");
Print("TRIPLE_TOTAL TESTED ",tried," ACCEPTED_BEFORE_FUSION ",acceptedBeforeFusion," CLASSES
",Length(triples),"\n");
Print("BOREL_ORDER_COUNTS ",Collected(List(borels,Size)),"\n");
Print("M22_EXHAUSTIVE_FINISHED\n");
QUIT;
```

phase164_m22_validate.g

```
# Post-enumeration witness verification. This does not prove exhaustiveness.
LoadPackage("tomlib");;
SizeScreen([100000,24]);;
Read("phase164_m22_witnesses.g");;
started:=Runtime();;
Check:=function(test,msg) if not test then Error(msg);fi;end;;
Check(Size(G)=443520,"wrong group order");;
tom:=TableOfMarks("M22");;R:=List(MaximalSubgroupsTom(tom)[1],i->RepresentativeTom(tom,i));;
allM:=Concatenation(List(R,H->Orbit(G,H,OnPoints)));;
AP:=Action(A,allM,OnPoints);;
Check(Size(AP)=Size(AutomorphismGroup(G)),"automorphism action incomplete");;
PairChecks:=0;;TripleChecks:=0;;HellyChecks:=0;;GenChecks:=0;;PanelChecks:=0;;
Signature:=function(t)
 return [SortedList(List(t,Size)),SortedList(List(Combinations(t,2),p->Size(Intersection(p))
 )),Size(Intersection(t))];
end;;
for family in [M22Pairs,M22Triples] do
 sets:=List(family,t->Set(List(t,H->Position(allM,H))));
 Check(ForAll(sets,s->not fail in s),"nonmaximal witness");
 sigs:=List(family,Signature);
 for i in [1..Length(family)] do
  t:=family[i];r:=Length(t);B:=Intersection(t);
  Check(Size(Group(Concatenation(List(t,GeneratorsOfGroup))))=Size(G),"disconnected global
  incidence");
  for j in [1..i-1] do
   if sigs[i]=sigs[j] then
    Check(RepresentativeAction(AP,sets[i],sets[j],OnSets)=fail,"duplicate automorphism/type
    orbit");
   fi;
  od;
  if r=2 then
   Check(ForAll(t,H->Index(H,B)>1),"rank-two panel not firm");PairChecks:=PairChecks+1;
  else
   for k in [1..3] do
    others:=Difference([1..3],[k]);j:=others[1];l:=others[2];
    P:=Intersection(t[k],t[j]);Q:=Intersection(t[k],t[l]);
```

```
    Check(Size(Group(Concatenation(GeneratorsOfGroup(P),GeneratorsOfGroup(Q))))=Size(t[k]),"
    disconnected vertex residue");GenChecks:=GenChecks+1;
    Rkl:=Intersection(t[j],t[l]);
    one:=Set(List(RightTransversal(t[j],P),g->RightCoset(t[k],g)));
    two:=Set(List(RightTransversal(t[l],Q),g->RightCoset(t[k],g)));
    Check(Length(Intersection(one,two))=Index(Rkl,B),"nontransitive
    chambers");HellyChecks:=HellyChecks+1;
    Check(Index(Rkl,B)>1,"rank-three panel not firm");PanelChecks:=PanelChecks+1;
   od;
   TripleChecks:=TripleChecks+1;
  fi;
 od;
od;
Print("WITNESSES_PAIRS ",PairChecks," TRIPLES ",TripleChecks," THREE_WAY_HELLY
",HellyChecks," RESIDUE_GENERATION ",GenChecks," FIRM_PANELS ",PanelChecks,"\n");
Print("BOREL_IDS ",Collected(List(M22Triples,t->IdGroup(Intersection(t)))),"\n");
Print("CPU_MILLISECONDS ",Runtime()-started,"\nM22_WITNESSES_VALIDATED\n");
QUIT;
```

orbit_coordinates.g

```
# Independent geometric decoding of the author witnesses; no source table input.
SizeScreen([100000,24]);;
if not IsBound(WITNESS_FILE) then Error("Set WITNESS_FILE explicitly");fi;
Read(WITNESS_FILE);;
moved22:=MovedPoints(G);;fixed:=Difference([1..24],moved22);;
if Length(moved22)<>22 or Length(fixed)<>2 then Error("wrong natural action");fi;
sourceReps:=maximalClassRepresentatives{[1,2,3,4,6,5,7,8]};;
shadowSizes:=[1,6,7,7,8,2,12,11];;
classes:=List(sourceReps,H->Orbit(G,H,OnPoints));;
allM:=Concatenation(classes);;types:=Concatenation(List([1..8],i->List(classes[i],H->i)));;
shadows:=[];;
for i in [1..Length(allM)] do
 choices:=List(Filtered(Orbits(allM[i],moved22),s->Length(s)=shadowSizes[types[i]]),Set);
 if (types[i]=8 and Length(choices)<>2) or (types[i]<>8 and Length(choices)<>1) then
 Error("wrong shadow multiplicity");fi;
 if not ForAll(choices,s->Stabilizer(G,s,OnSets)=allM[i]) then Error("shadow stabilizer
 mismatch");fi;
 Add(shadows,choices);
od;
families:=List([1..8],i->Set(Concatenation(shadows{Filtered([1..Length(allM)],j->types[j]=i)
})));;
blocks:=Set(Concatenation(List(families[2],s->Union(s,fixed)),List(families[3],s->Union(s,[f
ixed[1]])),List(families[4],s->Union(s,[fixed[2]])),families[5]));;
fives:=Collected(Concatenation(List(blocks,s->Combinations(s,5))));;
if Length(blocks)<>759 or Length(fives)<>Binomial(24,5) or not ForAll(fives,x->x[2]=1) then
Error("not Steiner S(5,8,24)");fi;
dodecads:=[];;
for i in [1..Length(blocks)] do
 for j in [i+1..Length(blocks)] do
  if Length(Intersection(blocks[i],blocks[j]))=2 then AddSet(dodecads,Union(Difference(block
  s[i],blocks[j]),Difference(blocks[j],blocks[i])));fi;
 od;
od;
if Filtered(dodecads,s->IsSubset(moved22,s))<>families[7] then Error("dodecad family
mismatch");fi;
endecads:=Set(Concatenation(List(fixed,f->List(Filtered(dodecads,s->f in s and
Length(Intersection(s,fixed))=1),s->Difference(s,[f])))));;
if endecads<>families[8] then Error("endecad family mismatch");fi;
Print("SHADOW_FAMILIES ",List(families,Length)," STEINER_BLOCKS ",Length(blocks),"
FIVE_SUBSETS ",Length(fives)," DODECADS ",Length(dodecads),"\n");

# Whole source definition allows both complementary endecad shadows.
# Canonical signatures additionally retain the older infinity extension.
L073Coordinates:=function(family,canonical)
 local output,idx,t,positions,ty,B,sigs,perm,sw,tt,inf,choices,pos,
       porders,ints,options,pp;
 output:=[];
 for idx in [1..Length(family)] do
  t:=family[idx];positions:=List(t,H->Position(allM,H));
  ty:=types{positions};B:=Intersection(t);sigs:=[];
  for perm in PermutationsList([1..Length(t)]) do
```

```
   for sw in [false,true] do
    tt:=ty{perm};inf:=fixed[2];
    if sw then tt:=List(tt,x->x^(3,4));inf:=fixed[1];fi;
    if tt<>SortedList(tt) then continue;fi;
    options:=[];
    for pos in positions{perm} do
     choices:=shadows[pos];
     if canonical and types[pos]=8 then
      choices:=Filtered(choices,s->Union(s,[inf]) in dodecads);
      if Length(choices)<>1 then Error("noncanonical endecad");fi;
     fi;
     Add(options,choices);
    od;
    pp:=Combinations([1..Length(t)],2);
    porders:=List(pp,p->Size(Intersection(t[perm[p[1]]],t[perm[p[2]]])));
    for choices in Cartesian(options) do
     ints:=List(pp,p->Length(Intersection(choices[p[1]],choices[p[2]])));
     AddSet(sigs,Concatenation(tt,ints,porders));
    od;
   od;
  od;
  Add(output,rec(witness:=idx,borel_id:=IdGroup(B),signatures:=sigs));
 od;
 return output;
end;;
L073Print:=function(path,family,canonical)
 local rows,i,r;
 rows:=L073Coordinates(family,canonical);
 PrintTo(path,"{\"fixed\":",fixed,",\"blocks\":",blocks,",\"triples\":[\n");
 for i in [1..Length(rows)] do
  r:=rows[i];if i>1 then AppendTo(path,",\n");fi;
  AppendTo(path,"{\"witness\":",r.witness,",\"borel_id\":",r.borel_id,",\"signatures\":",r.s
  ignatures,"}");
 od;
 AppendTo(path,"\n]}\n");
end;;
L073Print("canonical_triples.json",M22Triples,true);
L073Print("source_triples.json",M22Triples,false);
L073Print("canonical_pairs.json",M22Pairs,true);
L073Print("source_pairs.json",M22Pairs,false);
Print("L073_COMPLETE_COORDINATES ",Length(M22Pairs)," ",Length(M22Triples),"\n");
QUIT;
```

phase180_code_completions.py

```
#!/usr/bin/env python3
"""Enumerate all Golay-code completions of the literal primary MOG seeds.

This finite certificate retains the classical fact that Curtis's MOG comes
from a binary self-dual [24,12,8] Golay code. No QR labelling is assumed.
"""
import argparse
from collections import Counter
from hashlib import sha256
from itertools import combinations
import json
from pathlib import Path


def require(test, message):
    if not test:
        raise ValueError(message)


def insert(vector, basis):
    while vector:
        pivot = vector.bit_length() - 1
        if pivot in basis:
            vector ^= basis[pivot]
        else:
            basis[pivot] = vector
            return
```

```
def span(vectors):
    result = [0]
    for vector in vectors:
        result += [old ^ vector for old in result]
    require(len(set(result)) == len(result), "dependent span generators")
    return result


def support(vector):
    return [i for i in range(24) if vector >> i & 1]


def binding(path):
    data = path.read_bytes()
    return {"path": str(path), "bytes": len(data), "sha256": sha256(data).hexdigest()}


def complete(source):
    seeds = source["octad_seeds"]
    basis = {}
    for seed in seeds:
        points = seed["support"]
        require(len(points) == len(set(points)) == 8 and set(points) <= set(range(24)), "bad
        octad seed")
        insert(sum(1 << i for i in points), basis)
    require(len(basis) == 10, "primary seed code must have dimension10")
    require(all((a & b).bit_count() % 2 == 0 for a in basis.values() for b in
    basis.values()), "nonorthogonal primary seeds")
    lower = span(list(basis.values()))
    null_basis = []
    for free in sorted(set(range(24)) - set(basis)):
        vector = 1 << free
        for pivot in sorted(basis):
            if (vector & basis[pivot]).bit_count() % 2:
                vector ^= 1 << pivot
        require(all((vector & row).bit_count() % 2 == 0 for row in basis.values()),
        "nullspace equation")
        null_basis.append(vector)
    upper = span(null_basis)
    require(len(upper) == 2**14 and set(lower) <= set(upper), "orthogonal space dimensions")
    representatives, covered = [], set()
    for vector in upper:
        if vector not in covered:
            representatives.append(vector)
            covered.update(vector ^ word for word in lower)
    require(len(representatives) == 16, "quotient size")
    cosets = [{r ^ word for word in lower} for r in representatives]
    coset_of = {word: i for i, coset in enumerate(cosets) for word in coset}
    require(len(coset_of) == len(upper), "coset partition")
    allowed_weights = {0, 8, 12, 16, 24}
    allowed = {i for i, coset in enumerate(cosets) if all(word.bit_count() in allowed_weights
    for word in coset)}
    planes = set()
    for i, j in combinations(range(1, 16), 2):
        k = coset_of[representatives[i] ^ representatives[j]]
        planes.add(tuple(sorted([0, i, j, k])))
    require(len(planes) == 35 and all(len(set(p)) == 4 for p in planes), "all two-dimensional
    quotient subspaces")
    accepted = []
    for plane in sorted(planes):
        if not set(plane) <= allowed:
            continue
        words = set().union(*(cosets[i] for i in plane))
        require(len(words) == 4096, "full code size")
        require(Counter(w.bit_count() for w in words) == Counter({0: 1, 8: 759, 12: 2576, 16:
        759, 24: 1}), "Golay weight distribution")
        blocks = sorted(support(w) for w in words if w.bit_count() == 8)
        pentads = Counter(p for block in blocks for p in combinations(block, 5))
        require(len(pentads) == 42504 and set(pentads.values()) == {1}, "Steiner axiom")
        accepted.append({"quotient_plane": list(plane), "blocks": blocks,
                         "codeword_sha256": sha256(json.dumps(sorted(words), separators=(",",
                          ":")).encode()).hexdigest(),
```

```
                        "five_subset_count": len(pentads)})
    require(len(accepted) == 4, "unexpected compatible completion count")
    return {"seed_count": len(seeds), "seed_rank": 10, "orthogonal_rank": 14,
            "seed_basis_supports": [support(basis[k]) for k in sorted(basis)],
            "quotient_representative_supports": [support(r) for r in representatives],
            "quotient_subspaces_tested": len(planes), "allowed_cosets": sorted(allowed),
            "completions": accepted,
           "exhaustiveness": "Every binary self-dual dimension12 code containing the primary
            seed code lies in its orthogonal space and corresponds to one of the35
            two-dimensional quotient subspaces. Minimum8 and double evenness leave exactly
            these four. Each is separately verified to give S(5,8,24)."}


def main():
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--source", type=Path,
    default=Path(__file__).with_name("phase180_source_notation.json"))
    parser.add_argument("--target-shadows", type=Path, required=True)
    parser.add_argument("--output-dir", type=Path, required=True)
    args = parser.parse_args()
    require(not args.output_dir.exists() and not args.output_dir.is_symlink(), "output
    directory occupied")
    source = json.loads(args.source.read_text())
    target = json.loads(args.target_shadows.read_text())
    report = complete(source)
    report.update(status="PASS", source=binding(args.source),
    target_shadows=binding(args.target_shadows),
                  retained_input=source["retained_mathematical_input"],
                  QR_coordinate_identification_assumed=False)
    args.output_dir.mkdir()
    (args.output_dir / "code_completions.json").write_text(json.dumps(report, indent=2) +
    "\n")
    gap_data = {"sourceCodeBlocks": [[[p + 1 for p in block] for block in c["blocks"]] for c
    in report["completions"]],
                "targetBlocks": target["blocks"], "targetFixed": target["fixed"],
                "sourceTriples": [[[p + 1 for p in obj] for obj in case["objects"]] for case
                in source["colon_cases"]],
                "sourceLabels": [c["label"] for c in source["colon_cases"]],
               "sourcePairs": [[[p + 1 for p in obj] for obj in case["objects"]] for case in
                source["explicit_refined_pair_objects"]],
                "sourcePairLabels": [c["label"] for c in
                source["explicit_refined_pair_objects"]],
                "sourcePairTypes": [c["types"] for c in
                source["explicit_refined_pair_objects"]],
                "sourcePairOrders": [c["printed_pair_order"] for c in
                source["explicit_refined_pair_objects"]]}
    text = "\n".join(name + ":=" + json.dumps(value, separators=(",", ":")) + ";;" for name,
    value in gap_data.items()) + "\n"
    (args.output_dir / "code_inputs.g").write_text(text)
    print(json.dumps({"status": "PASS", "completions": 4, "seed_rank": 10,
    "quotient_subspaces_tested": 35}))


if __name__ == "__main__":
    main()
```

phase180_mog_check.g

```
# Check all compatible source Golay completions, not an assumed QR labelling.
# Set INPUT_FILE, WITNESS_FILE and OUTPUT_FILE before reading this program.
LoadPackage("design");;
SizeScreen([100000,24]);;
if not IsBound(INPUT_FILE) or not IsBound(WITNESS_FILE) or not IsBound(OUTPUT_FILE) then
 Error("three explicit file parameters required");
fi;
Read(INPUT_FILE);;
Read(WITNESS_FILE);;
Phase180Check:=function(test,msg) if not test then Error(msg);fi;end;;
Phase180Signature:=function(t)
 return [SortedList(List(t,Size)),SortedList(List(Combinations(t,2),p->Size(Intersection(p))
 )),Size(Intersection(t))];
end;;
```

```
Phase180Run:=function()
 local full, allM, AP, wsets, wsigs, tdesign, tgraph, c, sgraph, iso, p,
       case, pts, t, B, pairorders, res, helly, k, other, j, l, P, Q,
       one, two, flagtransitive, ids, tset, signature, i, first, maps,
       expectedOrders, actualSizes, memberships, sourceWords, sourceDodecads,
       octad, octad2;
 tdesign:=BlockDesign(24,targetBlocks);
 full:=AutGroupBlockDesign(tdesign);
 Phase180Check(Size(full)=244823040,"wrong full Steiner automorphism group");
 Phase180Check(Stabilizer(Stabilizer(full,targetFixed[1]),targetFixed[2])=G,"named M22 is
 not the pointwise two-point stabilizer");
 allM:=Concatenation(List(maximalClassRepresentatives,H->Orbit(G,H,OnPoints)));
 AP:=Action(A,allM,OnPoints);
 Phase180Check(Size(AP)=887040,"incomplete automorphism action");
 wsets:=List(M22Triples,t->Set(List(t,H->Position(allM,H))));
 wsigs:=List(M22Triples,Phase180Signature);
 tgraph:=IncidenceGraphSupportBlockDesign(tdesign);
 PrintTo(OUTPUT_FILE,"{\"status\":\"COMPLETE_EXACT_SOURCE_DIAGNOSTIC\",\"full_Steiner_automo
 rphism_order\":",Size(full),",\"M22_order\":",Size(G),",\"completion_cases\":[\n");
 first:=true;maps:=[];
 for c in [1..Length(sourceCodeBlocks)] do
  sgraph:=IncidenceGraphSupportBlockDesign(BlockDesign(24,sourceCodeBlocks[c]));
  iso:=GraphIsomorphism(
   rec(graph:=sgraph,colourClasses:=[[24],[15],Difference([1..24],[24,15]),[25..783]]),
   rec(graph:=tgraph,colourClasses:=[[targetFixed[2]],[targetFixed[1]],Difference([1..24],ta
   rgetFixed),[25..783]]));
  Phase180Check(iso<>fail,"source completion not isomorphic with distinguished points");
  p:=PermList(List([1..24],i->i^iso));
  Add(maps,p);
  Phase180Check(Set(List(sourceCodeBlocks[c],s->OnSets(s,p)))=Set(targetBlocks),"block
  isomorphism validation failed");
  for case in [1..Length(sourceTriples)] do
   pts:=List(sourceTriples[case],s->OnSets(Set(s),p));
   t:=List(pts,s->Stabilizer(G,s,OnSets));
   B:=Intersection(t);
   pairorders:=List([[1,2],[1,3],[2,3]],q->Size(Intersection(t[q[1]],t[q[2]])));
   res:=[];helly:=[];
   for k in [1..3] do
    other:=Difference([1..3],[k]);j:=other[1];l:=other[2];
    P:=Intersection(t[k],t[j]);Q:=Intersection(t[k],t[l]);
    Add(res,Size(Group(Concatenation(GeneratorsOfGroup(P),GeneratorsOfGroup(Q)))));
    one:=Set(List(RightTransversal(t[j],P),g->RightCoset(t[k],g)));
    two:=Set(List(RightTransversal(t[l],Q),g->RightCoset(t[k],g)));
    Add(helly,[Length(Intersection(one,two)),Index(Intersection(t[j],t[l]),B)]);
   od;
   flagtransitive:=ForAll(helly,x->x[1]=x[2]);
   ids:=[];
   if flagtransitive then
    tset:=Set(List(t,H->Position(allM,H)));
    Phase180Check(not fail in tset,"nonmaximal displayed object stabilizer");
    signature:=Phase180Signature(t);
    for i in [1..Length(M22Triples)] do
     if signature=wsigs[i] and RepresentativeAction(AP,tset,wsets[i],OnSets)<>fail then
     Add(ids,i);fi;
    od;
   fi;
   if not first then AppendTo(OUTPUT_FILE,",\n");fi;first:=false;
   AppendTo(OUTPUT_FILE,"{\"completion\":",c,",\"case\":",case,",\"label\":\"",sourceLabels[
   case],"\",\"point_map\":",List([1..24],i->i^p),",\"target_objects\":",pts,",\"stabilizer_
   orders\":",List(t,Size),",\"pair_orders\":",pairorders,",\"borel_id\":",IdGroup(B),",\"re
   sidue_generated_orders\":",res,",\"Helly_actual_expected\":",helly,",\"flag_transitive\":
   ",flagtransitive,",\"census_witnesses\":",ids,"}");
   Print("SOURCE_COMPLETION ",c," CASE ",case," HELLY ",helly," WITNESSES ",ids,"\n");
  od;
 od;
 AppendTo(OUTPUT_FILE,"\n],\"explicit_pair_cases\":[\n");first:=true;
 expectedOrders:=[20160,5760,2520,2520,1344,1920,720,660];
 for c in [1..Length(sourceCodeBlocks)] do
  p:=maps[c];
  sourceWords:=[[]];
  # Binary span of the octads, represented as supports.
  for octad in sourceCodeBlocks[c] do
   if not octad in sourceWords then
```

```
    sourceWords:=Set(Concatenation(sourceWords,List(sourceWords,w->Union(Difference(w,octad)
    ,Difference(octad,w)))));
   fi;
  od;
  Phase180Check(Length(sourceWords)=4096,"source octad span size");
  sourceDodecads:=Filtered(sourceWords,w->Length(w)=12);
  for case in [1..Length(sourcePairs)] do
   pts:=List(sourcePairs[case],s->OnSets(Set(s),p));
   t:=List(pts,s->Stabilizer(G,s,OnSets));
   actualSizes:=List(t,Size);
   memberships:=[];
   for i in [1,2] do
    if sourcePairTypes[case][i]=8 then
     Add(memberships,Union(sourcePairs[case][i],[24]) in sourceDodecads);
    elif sourcePairTypes[case][i]=7 then
     Add(memberships,Set(sourcePairs[case][i]) in sourceDodecads);
    elif sourcePairTypes[case][i]=5 then
     Add(memberships,Set(sourcePairs[case][i]) in Set(sourceCodeBlocks[c]));
    else Add(memberships,true);fi;
   od;
   if not first then AppendTo(OUTPUT_FILE,",\n");fi;first:=false;
   AppendTo(OUTPUT_FILE,"{\"completion\":",c,",\"pair\":",case,",\"label\":\"",sourcePairLab
   els[case],"\",\"canonical_object_membership\":",memberships,",\"stabilizer_orders\":",act
   ualSizes,",\"expected_stabilizer_orders\":",expectedOrders{sourcePairTypes[case]},",\"act
   ual_intersection_size\":",Length(Intersection(sourcePairs[case])),",\"actual_pair_id\":",
   IdGroup(Intersection(t)),",\"printed_pair_order\":",sourcePairOrders[case],"}");
   if not ForAll(memberships,x->x) or actualSizes<>expectedOrders{sourcePairTypes[case]} or
   Size(Intersection(t))<>sourcePairOrders[case] then
    Print("SOURCE_PAIR_DISCREPANCY ",c," ",sourcePairLabels[case],"
    membership=",memberships," actual=",actualSizes," pair=",IdGroup(Intersection(t)),"
    printed=",sourcePairOrders[case],"\n");
   fi;
  od;
 od;
 AppendTo(OUTPUT_FILE,"\n]}\n");
 Print("PHASE180_ALL_SOURCE_COMPLETIONS_CHECKED ",Length(sourceCodeBlocks),"\n");
end;;
Phase180Run();;
QUIT;
```

phase180_borel_check.g

```
# Check printed Borel shapes for the uniquely invariant-matched rows.
# A juxtaposition 2^a H / 3^a H is checked as a normal elementary-abelian
# subgroup with the stated quotient, without inferring an unspecified action.
if not IsBound(INPUT_FILE) or not IsBound(WITNESS_FILE) or not IsBound(OUTPUT_FILE) then
Error("explicit files required");fi;
Read(INPUT_FILE);;
Read(WITNESS_FILE);;
SizeScreen([100000,24]);;
Phase180BorelShape:=function(B,label)
 local named, extensions, row, p, rank, qid, N;
 named:=[["1",[1,1]],["2",[2,1]],["3",[3,1]],["4",[4,1]],
 ["22",[4,2]],["23",[8,5]],["24",[16,14]],["S3",[6,1]],
 ["D8",[8,3]],["Q8",[8,4]],["D10",[10,1]],["A4",[12,3]],
 ["D12",[12,4]],["F21",[21,1]],["S4",[24,12]],["A5",[60,5]],
 ["2× D8",IdGroup(DirectProduct(CyclicGroup(2),DihedralGroup(8)))]];
 row:=First(named,r->r[1]=label);
 if row<>fail then return IdGroup(B)=row[2];fi;
 extensions:=[["232",2,3,[2,1]],["322",3,2,[2,1]],["242",2,4,[2,1]],
 ["2222",2,2,[4,2]],["2322",2,3,[4,2]],["324",3,2,[4,1]],
 ["243",2,4,[3,1]],["22A4",2,2,[12,3]],["22D8",2,2,[8,3]],
 ["2422",2,4,[4,2]],["24A4",2,4,[12,3]]];
 row:=First(extensions,r->r[1]=label);
 if row=fail then Error("uninterpreted source Borel shape");fi;
 p:=row[2];rank:=row[3];qid:=row[4];
 if Size(B)<>p^rank*qid[1] then return false;fi;
 for N in NormalSubgroups(B) do
  if Size(N)=p^rank and IsElementaryAbelian(N) and IdGroup(FactorGroup(B,N))=qid then return
  true;fi;
 od;
 return false;
```

```
end;;
Phase180Borels:=function()
 local row,B,ok,first,failures;
 PrintTo(OUTPUT_FILE,"{\"status\":\"COMPLETE_SHAPE_DIAGNOSTIC\",\"extension_interpretation\"
 :\"Normal elementary-abelian kernel with printed quotient; no unspecified action or
 splitting inferred\",\"rows\":[\n");
 first:=true;failures:=[];
 for row in sourceBorelRows do
  B:=Intersection(M22Triples[row[2]]);ok:=Phase180BorelShape(B,row[3]);
  if not first then AppendTo(OUTPUT_FILE,",\n");fi;first:=false;
  AppendTo(OUTPUT_FILE,"{\"source_row\":",row[1],",\"witness\":",row[2],",\"printed_shape\":
  \"",row[3],"\",\"actual_small_group_id\":",IdGroup(B),",\"shape_compatible\":",ok,"}");
  if not ok then Add(failures,row[1]);fi;
 od;
 AppendTo(OUTPUT_FILE,"\n],\"incompatible_rows\":",failures,"}\n");
 Print("PHASE180_BOREL_SHAPES_COMPLETE ",Length(sourceBorelRows)," INCOMPATIBLE
 ",failures,"\n");
end;;
Phase180Borels();;
QUIT;
```

reconcile.py

```
#!/usr/bin/env python3
"""Separate the complete census from two explicit readings of the source labels."""
import argparse, collections, itertools, json, re
from pathlib import Path

def need(ok,msg):
    if not ok: raise ValueError(msg)
def read(p):return json.loads(Path(p).read_text())
def write(p,x):Path(p).write_text(json.dumps(x,indent=2,sort_keys=True)+'\n')
ORDERS={'A5':60,'1':1,'2':2,'3':3,'4':4,'22':4,'S3':6,'D8':8,'Q8':8,'23':8,'D10':10,'A4':12,
'D12':12,'24':16,'2×
D8':16,'232':16,'322':18,'F21':21,'S4':24,'242':32,'2222':16,'2322':32,'324':36,'243':48,'22
A4':48,'22D8':32,'2422':64,'24A4':192}
PAIRS=[(0,1),(0,2),(1,2)]

def index(data,n,arity):
    need(len(data['triples'])==n,'complete class list')
    need([w['witness'] for w in data['triples']]==list(range(1,n+1)),'unique ordered class
    IDs')
    inv={}
    for w in data['triples']:
        need(w['signatures'],'nonempty signature family')
        for s in w['signatures']:
            need(len(s)==arity,'signature arity')
           key=tuple(s);need(key not in inv or inv[key]==w['witness'],'signature separation')
            inv[key]=w['witness']
    return inv

def complements(signatures):
    sizes={1:1,2:6,3:7,4:7,5:8,6:2,7:12,8:11};out=set()
    for s in signatures:
        t=s[:3]
        for bits in itertools.product([False,True],repeat=t.count(8)):
            flips=dict(zip([i for i in range(3) if t[i]==8],bits));values=[]
            for k,(i,j) in enumerate(PAIRS):
                x=s[3+k];a=flips.get(i,False);b=flips.get(j,False)
                if a and b:x=22-sizes[t[i]]-sizes[t[j]]+x
                elif a:x=sizes[t[j]]-x
                elif b:x=sizes[t[i]]-x
                values.append(x)
            out.add(tuple(t+values+s[6:]))
    return out

def parse(label):
    m=re.fullmatch(r'M(\d{3})\((.*)\)',label);need(m,'label syntax')
    t=list(map(int,m[1]));body=m[2].split(':');tokens=[x.strip() for x in body[0].split(',')]
    need(len(body)<=2 and len(tokens)==3 and all(re.fullmatch(r'\d[12]?',x) for x in
    tokens),'intersection syntax')
```

```
    return t,[int(x[0]) for x in tokens],[int(x[1]) if len(x)>1 else None for x in
    tokens],int(body[1]) if len(body)==2 else None

def build(source,rows,broad,canonical,pairs,mog):
    inv=index(broad,431,9);cinv=index(canonical,431,9);pinv=index(pairs,81,4)
    for w,c in zip(broad['triples'],canonical['triples']):
        need(w['borel_id']==c['borel_id'] and complements(c['signatures'])=={tuple(s)for s
        in w['signatures']},'complete independent complement orbit')
    need([r['row'] for r in rows]==list(range(1,432)),'complete physical source rows')
    refined={(tuple(x['types']),x['intersection_size']):{int(k):v for k,v in
    x['subscript_orders'].items()} for x in source['refined_pairs']}
    explicit={};pair_receipts=[]
    for k,x in enumerate(source['explicit_refined_pair_objects'],1):
        cases=[y for y in mog['explicit_pair_cases'] if y['pair']==k]
        need(len(cases)==4 and {y['completion']for y in cases}=={1,2,3,4},'all four pair
        completions')
        vals={(tuple(y['canonical_object_membership']),tuple(y['stabilizer_orders']),tuple(y
        ['actual_pair_id']),y['actual_intersection_size']) for y in cases}
        need(len(vals)==1,'coordinate independent pair result')
        y=cases[0];valid=all(y['canonical_object_membership']) and
        y['stabilizer_orders']==y['expected_stabilizer_orders']
        key=tuple(x['types']+[y['actual_intersection_size'],y['actual_pair_id'][0]])
        pair_id=pinv.get(key) if valid else None
        need(not valid or pair_id is not None,'literal valid pair belongs to complete pair
        census')
        explicit[x['label']]={'valid':valid,'order':y['actual_pair_id'][0],'pair_class':pair
        _id,'provisional_e4':x['label']=='M88(7_1)'}
        pair_receipts.append({'label':x['label'],**explicit[x['label']],'printed_order':x['p
        rinted_pair_order'],'actual_pair_id':y['actual_pair_id']})
    colon={}
    for k,x in enumerate(source['colon_cases'],1):
        cases=[y for y in mog['completion_cases'] if y['case']==k]
        need(len(cases)==4 and len({(y['flag_transitive'],tuple(y['census_witnesses'])) for
        y in cases})==1,'coordinate independent colon result')
        colon[re.sub(r'[\s_]+','',x['label'])]=cases[0]['census_witnesses']
    byid={w['witness']:w for w in broad['triples']};result=[];borels=set()
    for row in rows:
        need(row['Borel_text'] in ORDERS,'interpreted printed Borel shape');t,v,sub,col=pars
        e(row['label_text']);porders=[];eorders=[];undefined=[];provisional=[]
        for k,(i,j) in enumerate(PAIRS):
            if sub[k] is None:porders.append(None);eorders.append(None);continue
            porders.append(refined[(tuple([t[i],t[j]]),v[k])][sub[k]])
            label=f'M{t[i]}{t[j]}({v[k]}_{sub[k]})';obj=explicit[label]
            eorders.append(obj['order'] if obj['valid'] else None)
            if not obj['valid']:undefined.append(label)
            if obj['provisional_e4']:provisional.append(label)
        def candidates(idx,orders):
            return sorted({w for s,w in idx.items() if list(s[:6])==t+v and all(o is None or
            s[6+k]==o for k,o in enumerate(orders))})
        variants={'canonical_pair_order':candidates(cinv,porders),'source_pair_order':candid
        ates(inv,porders),'explicit_pair_objects':None if undefined else
        candidates(inv,eorders)}
        # A colon is an actual displayed triple, not just its numeric shadow label.
        if col is not None:
            label=re.sub(r'[\s_]+','',row['label_text']);need(label in colon,'all colon
            labels transcribed')
            variants['explicit_pair_objects']=colon[label]
        for ids in variants.values():
            if ids is not None:
                for w in ids:borels.add((row['row'],w,row['Borel_text']))
        result.append({**row,'pair_orders_p112':porders,'literal_pair_orders_p114':eorders,'
        undefined_literal_pair_labels':undefined,'provisional_literal_pair_labels':provision
        al,'candidates':variants,'candidate_Borel_ids':{str(w):byid[w]['borel_id'] for ids
        in variants.values() if ids is not None for w in ids}})
    return {'status':'COMPLETE_CORRECTED_CENSUS_WITH_SEPARATE_SOURCE_READINGS','pair_classes
    ':81,'triple_classes':431,'separating_pair_signatures':len(pinv),'separating_triple_sign
    atures':len(inv),'canonical_triple_signatures':len(cinv),'source_rows':result,'literal_p
    air_receipts':pair_receipts,'replacement_pairs':pairs['triples'],'replacement_triples':c
    anonical['triples'],'source_complement_policy':'Every endecad may be replaced
    independently by its complement, as permitted in L073 pp2–3. Canonical signatures are a
    separate table convention.','historical_geo_numbering_recovered':False,'unconditional_al
    l_source_labels_matched':False},sorted(borels)
```

```
def finish(report,shapes):
    verdict={(r['source_row'],r['witness']):r['shape_compatible'] for r in shapes['rows']}
    summaries={}
    for variant in ['canonical_pair_order','source_pair_order','explicit_pair_objects']:
        assignments=collections.defaultdict(list);bad=[];undefined=[]
        for row in report['source_rows']:
            ids=row['candidates'][variant]
            if ids is None:undefined.append(row['row']);continue
            good=[i for i in ids if verdict[row['row'],i]]
            row.setdefault('shape_compatible',{})[variant]=good
            if not good:bad.append(row['row'])
            for i in good:assignments[i].append(row['row'])
        summaries[variant]={'compatible_rows':sum(len(v) for v in
        assignments.values()),'distinct_classes':len(assignments),'incompatible_rows':bad,'u
        ndefined_rows':undefined,'collisions':[{'class':i,'rows':rs} for i,rs in
        sorted(assignments.items()) if
        len(rs)>1],'unassigned_classes':sorted(set(range(1,432))-set(assignments))}
    report['summaries']=summaries;return report

def main():
    p=argparse.ArgumentParser();p.add_argument('--data',type=Path,required=True);p.add_argum
    ent('--work',type=Path,required=True);p.add_argument('--finish',action='store_true');a=p
    .parse_args()
    out=a.work
    if a.finish:
        x=finish(read(out/'reconciliation_pre.json'),read(out/'Borel_check.json'));write(out
        /'reconciliation.json',x);print(json.dumps(x['summaries'],sort_keys=True));return
    x,b=build(read(a.data/'phase180_source_notation.json'),read(a.data/'phase164_L073_table_
    contract.json')['rows'],read(out/'source_triples.json'),read(out/'canonical_triples.json
    '),read(out/'source_pairs.json'),read(out/'codes/mog_check.json'))
    write(out/'reconciliation_pre.json',x);(out/'Borel_inputs.g').write_text('sourceBorelRow
    s:='+json.dumps(b,ensure_ascii=False)+';;\n');print('L073_RECONCILIATION_PREPARED',len(b
    ))
if __name__=='__main__':main()
```